Failure (a real rollback trigger) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The shape of this exercise is: keep a running list of scores, and after every new one arrives, look at just the last `window_size` of them (a "sliding window") and compare their average against a baseline — the score you'd expect a healthy version to keep hitting. If the recent average has fallen far enough below the baseline — say, dropped by 20% or more — that's the trigger firing.

Percent drop is `(baseline - avg_recent) / baseline`. A `baseline` of `0.95` and a recent average of `0.60` gives `(0.95 - 0.60) / 0.95 ≈ 0.37`, a 37% drop — well past a `0.2` (20%) threshold.

The exact pieces:

- `scores_so_far[-window_size:]` — Python's negative-index slicing grabs the last `window_size` items of a list, however many there are (it won't error even if the list is shorter than `window_size`, which matters for the next point).
- A guard for not enough data yet: `if len(scores_so_far) < window_size: return False` — you can't trust a 2-score average to represent "the last 5 requests" if only 2 have happened.
- `percent_drop = (baseline - avg_recent) / baseline`, then `percent_drop >= drop_threshold`.
- A `rollback_to_previous()` function — for this exercise, a stub that just prints what it would do and returns the version name it reverted to; in the Build Task, this is the real function that rewrites the live pointer.

Hint 2 — The plan, and almost the whole thing

```
function should_rollback(scores_so_far: list[float], baseline: float,
                         window_size: int,
                         drop_threshold: float) -> bool:
```

```
    if not enough scores yet: return False
    recent = last window_size scores
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

function rollback_to_previous() -> str:
    print what happened
    return the version name rolled back to
```

```
# rollback_trigger_practice.py
def should_rollback(
    scores_so_far: list[float],
    baseline: float,
    window_size: int = 5,
    drop_threshold: float = 0.2,
) -> bool:
    if len(scores_so_far) < window_size:
        return False
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

def rollback_to_previous() -> str:
    print("ROLLBACK: reverting to last known-good version")
    return "v1"
```

Now write the loop that feeds a simulated sequence of scores through `should_rollback`, and notice it keeps calling `rollback_to_previous()` on every request after the first trigger — the repeat-firing problem Hint 1's Intermediate section named. Compare against the Solution once you've seen it happen yourself.

Full solution: Show me the solution